

Whistleblower Website Encryption Implementation Guide (AES-256-GCM + RSA)

1. Goal of the Architecture (What We Are Building)

The objective is to ensure:

Reports are encrypted **inside the whistleblower's browser** before leaving the device.

Backend server cannot decrypt reports even if hacked.

Database stores only encrypted data (ciphertext).

Only authorised investigators holding the private key can decrypt reports.

This is a typical **Zero-Knowledge / End-to-End Encryption** model.

2. Where the Encryption Must Be Implemented

A. Frontend (Browser) – Must Perform Encryption

This is the most important part.

Encryption happens in the browser using JavaScript before the report is uploaded.

Implement in:

- Report submission form (HTML/React/Vue)
- Client-side JavaScript encryption module
- File upload encryption logic (if attachments exist)

B. Backend (Server) – Stores Data Only (No Decryption)

Backend must act like a transport + storage system only.

Implement in:

- API endpoints (POST /submitReport)
- Database schema storing ciphertext
- Metadata logging (minimal)

Backend must **never have access to private keys**.

C. Investigator Portal – Performs Decryption

Investigator downloads encrypted report and decrypts locally.

Implement in:

- Secure investigator dashboard
- Private key storage (hardware token or local secure storage)
- Client-side decryption module

3. Cryptography Design (What Exactly Happens)

Step 1: Generate AES Key (Symmetric Encryption Key)

The browser generates a random AES key:

- AES key size: **256-bit**
- Algorithm: **AES-GCM**
- Why: fast encryption and includes integrity protection

AES Key is used to encrypt the report itself.

Step 2: Encrypt the Report Using AES-256-GCM

AES-GCM produces:

- ciphertext
- IV (nonce)
- authentication tag (integrity proof)

Important: GCM requires a unique IV per encryption.

Step 3: Encrypt AES Key Using RSA Public Key

Since AES key is temporary, it must be protected.

The browser encrypts the AES key using the investigator's RSA public key.

- RSA size: minimum **2048-bit** (recommended 3072-bit)
- Padding: **RSA-OAEP with SHA-256**

This ensures only the investigator can recover the AES key using the private key.

4. Data Sent to Backend (What Backend Receives)

Backend receives a payload like:

- encryptedReport (ciphertext)
- encryptedAESKey (RSA encrypted key)
- iv (nonce for AES-GCM)
- tag (auth tag, often included in ciphertext depending on implementation)
- optional: reportID, timestamp, submission category

Backend does **NOT** receive plaintext.

5. Database Storage Requirements

Database stores:

Field	Description
report_id	UUID
encrypted_report	AES ciphertext
encrypted_aes_key	RSA encrypted AES key
iv	AES-GCM IV
created_at	timestamp
status	open/closed
metadata	minimal, non-sensitive

Do not store whistleblower IP addresses unless legally required.
If required, store separately with strong access control.

6. Investigator Decryption Process

Investigator workflow:

1. Download encrypted report from backend
2. Decrypt AES key using RSA private key
3. Decrypt report using AES key and IV
4. Plaintext appears only inside investigator environment

7. Recommended Coding and Tools

A. Frontend Encryption (Browser)

Use **Web Crypto API** (industry standard).

Why Web Crypto API?

- Built into modern browsers
- Secure random generation
- Avoids insecure custom crypto libraries
- FIPS-aligned implementations in many environments

Developer implementation:

- JavaScript module: `crypto.js`
- Function-based structure:
 - `generateAESKey()`
 - `encryptReportAES(report, key)`
 - `encryptAESKeyRSA(aesKey, rsaPublicKey)`
 - `submitEncryptedPayload()`

B. Backend (API + Database)

Backend should be simple REST.

Recommended stack:

- Node.js (Express) / Python (FastAPI) / Java (Spring Boot)

Backend role:

- Accept encrypted payload

- Validate payload structure
- Store ciphertext as Base64 or binary (BLOB)
- Provide download endpoint

Backend must NOT run any decrypt function.

C. Investigator Decryption

Decryption can be implemented in:

Option 1 (Recommended):

- Investigator web portal using Web Crypto API
- Private key imported securely (manual upload or device key store)

Option 2:

- Offline investigator desktop tool (strongest security)
- Python tool using cryptography library or OpenSSL

8. Public Key Handling (Critical Implementation Detail)

Where the RSA Public Key Comes From

The browser must retrieve the investigator's public key securely.

Best practice:

- Hardcode public key fingerprint into frontend build
- Or host public key at endpoint: /publickey.pem

Security requirement:

The public key must be integrity-protected, otherwise attacker can replace it.

Best approach:

- Pin the public key fingerprint in frontend code

- Use HTTPS + CSP + SRI

9. Example Implementation Flow (Frontend)

Whistleblower Submission Page Logic

In submitReport.js

1. User types report text / uploads file
2. Browser generates AES key
3. Encrypt report with AES-GCM
4. Encrypt AES key using RSA public key
5. Send ciphertext + encrypted key to backend via HTTPS POST

10. File Upload Support (Attachments)

Attachments must follow the exact same pattern:

- Generate AES key once per report (or per file)
- Encrypt file bytes using AES-GCM
- Encrypt AES key using RSA-OAEP
- Upload ciphertext

Store each file ciphertext separately.

11. Security Hardening Requirements (Professional Standard)

A. Ensure Unique IV Per Encryption

AES-GCM breaks if IV is reused with same key.

- Generate 12-byte random IV (crypto.getRandomValues())

B. Use Content Security Policy (CSP)

To prevent injected JavaScript stealing plaintext before encryption.

Example CSP:

- Disallow inline scripts
- Only allow scripts from trusted domain

C. Protect Against XSS (Very Critical)

If attacker injects JavaScript, they can capture plaintext before encryption.

So implement:

- input sanitisation
- strict CSP
- output encoding
- disable unsafe inline scripts

D. Do Not Log Sensitive Data

Backend logging must never store:

- plaintext
- decrypted content
- report submission form values

E. Key Management Rules

- RSA private key must never be on backend server
- Only investigators keep private key
- Store private key in:
 - Hardware security key (best)
 - Encrypted disk vault
 - OS secure keystore

12. Professional System Summary (Developer-Friendly)

Client Side (Browser)

Technology: JavaScript + Web Crypto API

Responsibilities:

- Generate AES-256 key
- Encrypt report with AES-GCM
- Encrypt AES key with RSA-OAEP
- Upload encrypted payload

Server Side (Backend)

Technology: Node.js/Python/Java + Database

Responsibilities:

- Accept ciphertext payload
- Store ciphertext
- Allow download for investigators
- Never decrypt

Investigator Side

Technology: Web Portal or Offline Tool

Responsibilities:

- Download ciphertext
- Decrypt AES key using RSA private key
- Decrypt report using AES key
- View plaintext securely

13. Deliverable Developer Checklist (Implementation Must-Have)

Frontend

Web Crypto API AES-GCM encryption

RSA-OAEP encryption of AES key

Public key retrieval + pinning

Upload ciphertext only

Secure IV generation

Encrypt attachments

Backend

Database stores only ciphertext
No private key storage
No decryption code
Secure authentication for investigator portal

Investigator

Private key import
AES key recovery
Decrypt report locally
Access control and audit logs